



**Kingdom of Saudi Arabia
Ministry of Education
King Faisal University
College of Computer Sciences
& Information Technology**



Library Management System

**Course:
Data Structures and Algorithms**



Group: E

Teame Members

- Reda Alqatifi [ID]
- Ali Alburahim [ID]
- Mohammed Aljasem [ID]
- Mahdi Alhashim [ID]

Supervised by

- Dr. Sami Abdulsalam Alnawali

Date: December 2025

Project Overview

Introduction:

- ▶ A console-based application developed in C++ to manage library operations efficiently.
- ▶ Focuses on manipulating dynamic data using pointers rather than fixed arrays.

Key Objectives:

- ▶ Implement custom Data Structures from scratch (No STL containers).
- ▶ Apply efficient Sorting and Searching algorithms.
- ▶ Manage Dynamic Memory Allocation to optimize performance.
- ▶ Analyze Time and Space Complexity of the implemented solutions.

Data Structures Implemented

Doubly Linked List (For Books)

- ▶ Structure: Each node contains Book Data + Next Pointer + Previous Pointer.
- ▶ Why? Allows bidirectional traversal and efficient deletion $O(1)$ when the node pointer is known.

Singly Linked List (For Sections)

- ▶ Structure: Node contains Section Name + Next Pointer.
- ▶ Why? Memory efficient and sufficient for linear categories display.

Algorithm Used: Merge Sort

- **The Divide and Conquer Approach:** We selected Merge Sort as our primary sorting engine. Operating on the "Divide and Conquer" principle, the algorithm recursively splits the book list into smaller sub-lists until they are atomic. It then reconstructs (merges) them back together in the correct order. This recursive approach is particularly elegant and effective for Linked Lists.
- **Performance & Suitability:** Our choice was driven by the specific nature of our data structure. Unlike Arrays, Linked Lists do not support random access, which makes algorithms like Quick Sort less efficient here. Merge Sort, however, excels with sequential access patterns. It guarantees a stable time complexity of $O(N \log N)$, making it vastly superior to basic algorithms like Bubble Sort $O(N^2)$ for handling large datasets.

Algorithms: Searching

- **Algorithm Used:** Linear Search (Smart Implementation)Method: Iterates through nodes to match user input.
- **Enhanced Features:**
 - Partial Matching: Can find "Potter" using the keyword "Pot".
 - Case Insensitivity: Converts input and data to lowercase before comparison.
- **Complexity:** Worst-case Time Complexity of $O(N)$.

Complexity Analysis

Operation	Algorithm / Method	Time Complexity
Sorting	Merge Sort	$O(N \log N)$
Searching	Linear Search	$O(N)$
Insertion	Traversal to End	$O(N)$
Deletion	Search & Remove	$O(N)$
Space	Dynamic Nodes	$O(N)$

Note: Space complexity is efficient as memory is allocated only when a book is added.

Implementation Challenges

Memory Management

One of the primary challenges in C++ is manual memory management. To prevent memory leaks, we implemented a strict garbage collection routine (`clearMemoryLeak`) that ensures every node is properly deleted and memory is freed when the program terminates.

Robust Input Validation

To ensure the system never crashes due to invalid user input (e.g., entering text in a price field), we built a dedicated error-handling layer. This functions as a filter, checking all inputs before processing them.

System Execution And Interface

Main Menu Interface.

```
-----  
<<<<<<<[  Welcome to our Library Management System  ]>>>>>>>  
-----  
  
=====
```

Main Menu	
1 - Add a new (book / section).	
2 - Remove a (book / section).	
3 - Update a (book / section).	
4 - Sort the books.	
5 - Search a (book / section).	
6 - display (books / sections).	
7 - Total (books / sections).	
8 - Exit.	

```
=====
```

System Execution And Interface

Adding Sections or Books:

```
=====
|                               Add Menu                               |
|-----|
| 1 - Add a new book.          |
| 2 - Add a new section.       |
| 3 - Back to the Main Menu.   |
|-----|
|                               |
> Enter your choice : 1
|-----|

Enter The Book number [ 1 ] details.

> Title : Data Structures
> Author : The Dr
> Code : 223344
> Section ( 0 - Neither , 1 - Fiction , 2 - Acadimic , 3 - Business ) - (Write a Number) : 2
> Price ($) : 19
> Availible Quantity : 7

Do you want to add another book ( yes / no ) ? no
Ok!
|-----|
```

System Execution And Interface

Search Results Display:

```
=====
|                               |
|           Books Search Menu |
|                               |
| 1 - Search by { Title }.    |
| 2 - Search by { Code }.     |
| 3 - Search by { Author }.   |
| 4 - Search by { Section }.  |
| 5 - Back to the Main Menu.  |
|                               |
=====

> Enter your choice : 1

> Enter the Title to search : data

=====
| No. | Title           | Author   | Code   | Section | Price  | Quantity |
=====
| 1   | Data Structures | The Dr   | 223344 | Acadimic | $19.00 | 7         |
=====
```



Conclusion

Summary Statement: This project served as a practical application of core Data Structures. By building Linked Lists from scratch, we moved away from the limitations of fixed arrays and embraced dynamic memory allocation. The result is a modular, efficient, and stable Library Management System.

Thank You

Any Questions?